MERIDIAN

Live Trading Dashboard with Integrated AI Chatbot

Development Documentation

Prepared: August 2026

1. Project Overview

MERIDIAN is a browser-based, single-file live market dashboard, built to satisfy the practical component of an MSc Trading research project. It brings together three layers: live market data and charting sourced from TradingView's official embeddable widgets, a transparent, rule-based technical signal engine computed client-side from live crypto data, and a conversational AI chatbot powered by Google Gemini that can discuss the live signal in natural language.

The project brief called for the dashboard to “integrate an AI chatbot.” This document records the reasoning, architecture, and iterative build process behind the final implementation, including the technical obstacles that came up along the way and how each was resolved.

2. System Architecture & Technology Stack

The entire application lives in a single HTML file, trading_dashboard.html, with CSS and JavaScript embedded directly inside it — there is no build step, server, or database. That was a deliberate choice: it keeps the project runnable by simply opening the file in a browser, or by deploying it to free static hosting with no configuration required.

Layer	Details
Charting & live quotes	TradingView official embed widgets (Advanced Chart, Ticker Tape, Market Overview, Technical Analysis, Symbol Info, Economic Calendar) — free, no API key, real-time exchange data.
Live candle data (signal engine)	Binance public REST API (api.binance.com/api/v3/klines) — free, key-less, crypto pairs only.
AI chatbot	Google Gemini API (generativelanguage.googleapis.com), called directly from the browser using the user's own free API key.
Front end	Plain HTML5, CSS3 (custom properties for theming), vanilla JavaScript — no framework.
Deployment	Static hosting (e.g. Netlify Drop) — no backend required.

3. Development Process

The dashboard was built iteratively, in six stages, each corresponding to a distinct milestone.

3.1 Stage 1 — Live Charting Dashboard

The first version assembled six official TradingView widgets into a single-page layout: a scrolling ticker tape spanning crypto, equities, forex, and commodities; a large focus chart; a tabbed market overview; a

technical-analysis gauge; a symbol-info panel; and an economic calendar. Buttons let the user switch the "focus symbol" across asset classes, which rebuilds the chart, gauge, and symbol-info widgets for the newly selected asset.

3.2 Stage 2 — Rule-Based AI Signal Engine

To add genuine analytical capability beyond simply displaying charts, a signal engine was written in vanilla JavaScript. It fetches live 15-minute candles from Binance's public API for the focused crypto symbol, then computes three classic technical indicators client-side:

- RSI (14-period Relative Strength Index) — momentum / overbought-oversold
- MACD (12, 26, 9) — trend momentum via moving-average crossover
- SMA 50 / SMA 200 — long-term trend direction ("golden cross" logic)

Each indicator casts a weighted vote between -100 and +100. The votes are summed into a composite score, which maps to a BUY / SELL / HOLD verdict with an associated confidence percentage. This approach was chosen specifically because it's fully explainable: every number driving the verdict can be shown and justified, which matters for academic defensibility — unlike an opaque trained model, where a verdict can't easily be traced back to a reason.

3.3 Stage 3 — Visual Design Iteration

The interface went through two theme changes before settling on its current look: an initial dark "trading-terminal" theme (graphite background, amber and cyan accents modelled on Bloomberg-style terminals), followed by a "deep-space" dark theme (navy and violet gradients with a starfield effect) once the AI chat agent was added, and finally a clean white theme for better readability and a more conventional dashboard look. Each change required updating not just the page's own CSS but also the colorTheme parameters passed into the TradingView widgets themselves, since those render as embedded iframes with their own independent dark/light setting.

3.4 Stage 4 — Conversational AI Chatbot Integration

A chat panel was added and wired to a real large language model, rather than a scripted response system, to genuinely satisfy the "integrate an AI chatbot" requirement. Two providers were evaluated:

- Google Gemini API — free tier, no billing required, and confirmed to support direct cross-origin (CORS) calls from a browser, making it viable for a static, backend-less application.
- OpenAI (ChatGPT) API — requires a funded billing account even for minimal use, and has inconsistent CORS support for direct browser calls, which risks a request being blocked outright with no client-side fix available.

Gemini was selected as the production integration for these practical reasons. The chat agent is context-aware: every message automatically includes the currently focused symbol and the live signal engine's verdict, score, confidence, and indicator breakdown, so the model's answers stay grounded in the actual data on screen rather than defaulting to generic commentary.

3.5 Stage 5 — Reliability Engineering

Three distinct failure modes came up during testing, and each was fixed at the code level rather than worked around by hand:

Failure	Cause	Resolution
HTTP 404 — model retired	Google deprecated gemini-2.5-flash for newly created API keys mid-project.	Switched the default model to gemini-3.5-flash and added an automatic fallback list (gemini-3.5-flash-lite, gemini-2.5-flash-lite), so a future model retirement degrades gracefully instead of breaking the app.
HTTP 503 — server overload	“High demand” transient errors from Google's inference servers.	Added a short automatic retry (1.5s backoff) before falling through to the next candidate model.
HTTP 400 — invalid key	User-side key copy/paste errors, or a key generated for the wrong Google product.	No code fix is possible here, since the key is genuinely invalid — documented a troubleshooting checklist instead: regenerate via the copy icon, confirm the key came from Google AI Studio specifically, then reconnect.

3.6 Stage 6 — Deployment

The finished file was deployed to a public URL using Netlify Drop, a free static-hosting service that needs no account and no command line — the HTML file is dragged directly onto the Netlify Drop web page, which returns a live https:// link immediately. Because the application is entirely client-side, no server configuration was necessary; each visitor supplies their own Gemini API key, which is never transmitted to or stored by the hosting provider.

4. AI Integration — Technical Detail

4.1 Signal Engine Scoring Logic

A simplified excerpt of the composite scoring function:

```
const score = rsiVote + macdVote + trendVote;   // clamped to [-100, 100]

if (score >= 20)       verdict = "BUY";
else if (score <= -20) verdict = "SELL";
else                   verdict = "HOLD";

confidence = Math.min(100, Math.abs(score) + 20);
```

4.2 Gemini Chat Call with Context Injection

A simplified excerpt showing how live signal data is injected into every chat request as a system instruction:

```
function buildSystemContext() {
  const { label, result } = currentSignalContext;
  return `Focused symbol: ${label}. Engine verdict: ${result.verdict}, ` +
         `score ${result.score}, confidence ${result.confidence}%.`;
}

fetch(`.../models/${model}:generateContent?key=${apiKey}`, {
  method: "POST",
```

```
  body: JSON.stringify({
    system_instruction: { parts: [{ text: buildSystemContext() }] },
    contents
  })
});
```

The complete, unabridged source code for both the signal engine and the chat integration is contained in trading_dashboard.html, submitted alongside this document.

5. Feature Summary

Feature	Description
Ticker Tape	Live scrolling quotes across crypto, equities, forex, gold, oil, and indices.
Advanced Chart	Full TradingView charting for the focused symbol with switchable timeframes.
Market Overview	Tabbed live snapshot by asset class (crypto / equities / forex / futures).
Technical Gauge	TradingView's own buy/sell/neutral aggregation for the focused symbol.
Symbol Info	Key stats (price, volume, market cap where applicable) for the focused symbol.
Economic Calendar	Upcoming macroeconomic events filtered by major economies.
AI Signal Engine	Rule-based RSI/MACD/SMA verdict with confidence score, refreshed every 30 seconds.
Gemini Chat Agent	Context-aware conversational AI that explains the live signal on request.

6. Limitations & Future Work

- The signal engine only covers crypto pairs, since Binance is the only free, key-less live-candle source used; extending it to stocks or forex would require a keyed provider (e.g. Twelve Data, Alpha Vantage) or a broker API.
- The signal engine is rule-based rather than a trained model. A future iteration could train a classifier — gradient-boosted trees or an LSTM, for example — on historical OHLCV data and compare its predictive accuracy against this rule-based baseline.
- The Gemini API key must be re-entered each session by design, since it is never persisted — a deliberate security trade-off appropriate for a shared, public deployment.